

University OF Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	Course Code: CMPE-112L
Assignment Type: lab	Dated: 20 APRIL 2026
Semester: 1st	Session: Spring 2026
Lab/Project/Assignment #: 6	CLOs to be covered: CLO2
Lab Title: Local and Global Variables	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals/Objectives:

1. Objective of Local Variables

- **Data Encapsulation & Privacy:** The primary objective is to keep variables isolated. A function's internal variables (like `fact = 1` in your factorial task or `gp` and `ch` in your GPA loops) are shielded from the rest of the program, preventing outside code from accidentally changing their values.
- **Memory Efficiency (Short Lifetime):** Local variables only exist while the function is actively running. As soon as the function reaches a `return` statement or finishes executing, its local variables are destroyed, freeing up system memory.
- **Preventing Name Collisions:** By localizing variables, you can safely use common names (like `i`, `x`, or `total`) in ten different functions without worrying about them interfering with or overwriting one another.

2. Objective of Global Variables

- **State Sharing:** The main objective is to hold data that needs to be shared across multiple separate functions, modules, or systems without having to constantly pass the data back and forth as function arguments.
- **Persistent Lifetime:** Unlike local variables, global variables remain alive in memory for the entire duration of the program's execution, making them useful for tracking configurations, application settings, or system-wide states

Equipment Required:

- Computer/Laptop
- Python 3.x
- VS Code / PyCharm / IDLE

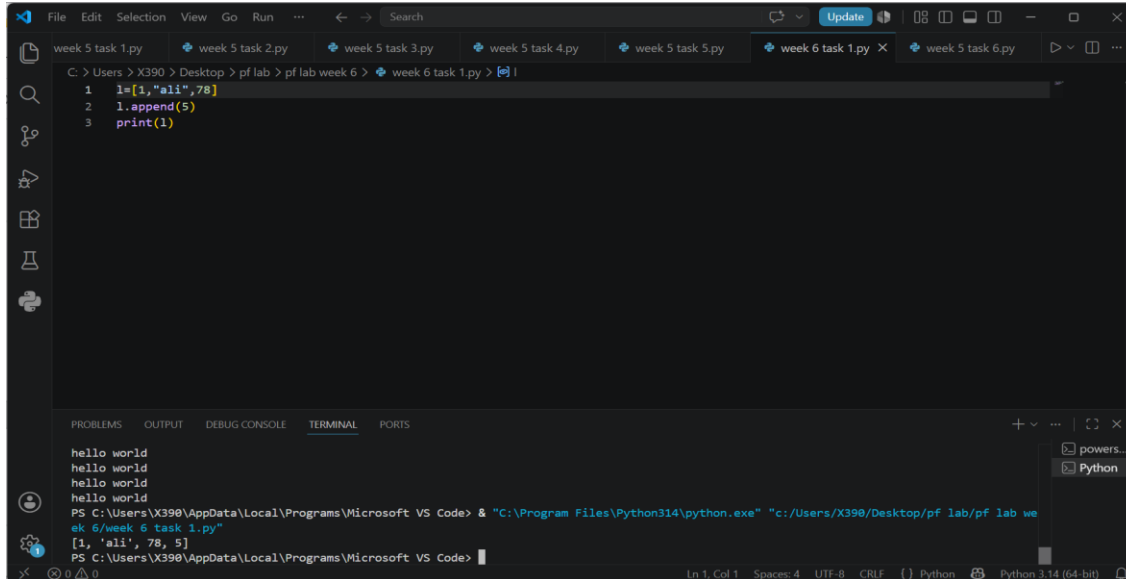
THEORY:

The theory of variable scope dictates how data is isolated and accessed within a program, dividing variables into distinct local and global categories. Local variables are declared strictly inside a function, encapsulating data to protect it from being accidentally modified by external code. This localized structure ensures memory efficiency because these variables only exist during the function's execution lifecycle and are destroyed immediately upon returning a value. Conversely, global variables are defined outside all functions at the script level, remaining alive for the program's entire duration to share application-wide configurations or persistent states. Striking a careful balance between these two scopes prevents naming collisions, simplifies debugging, and keeps complex software codebases modular and maintainable.

TASK NO 1:

Write a program that demonstrates the mutability and dynamic growth of Python list collections.

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 6 task 1.py' open. The code in the editor is:

```
1 l=[1,"ali",78]
2 l.append(5)
3 print(l)
```

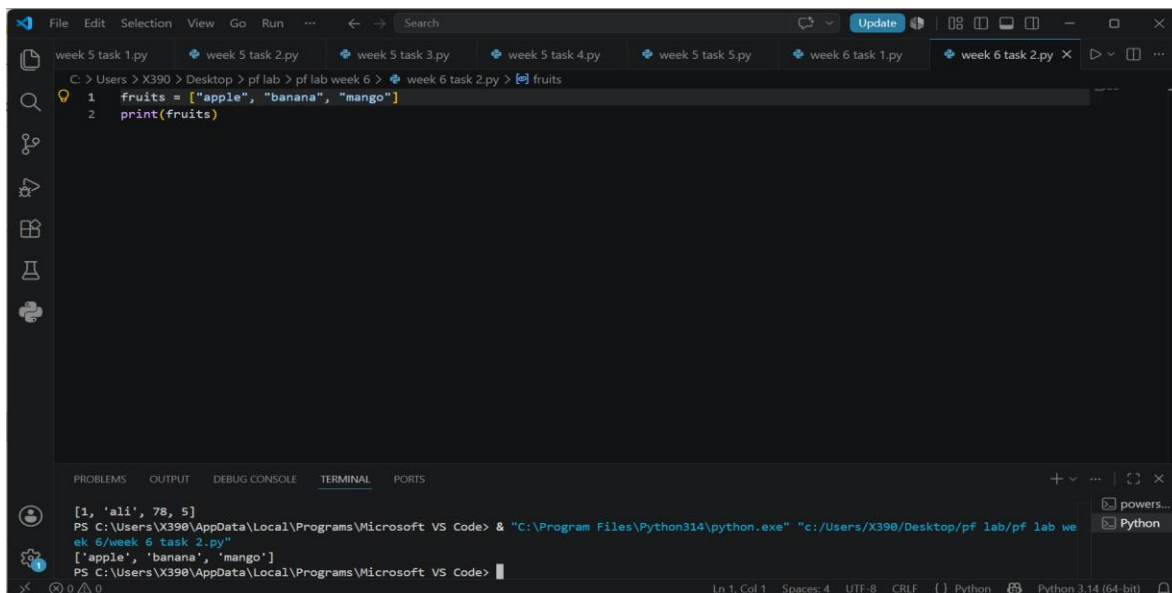
The terminal at the bottom shows the output of the script:

```
hello world
hello world
hello world
hello world
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf_lab/pf_lab we
ek 6/task 1.py"
[1, 'ali', 78, 5]
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 2:

Write a program that demonstrates how to declare and display a basic homogeneous list collection in Python

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 6 task 2.py' open. The code in the editor is:

```
1 fruits = ["apple", "banana", "mango"]
2 print(fruits)
```

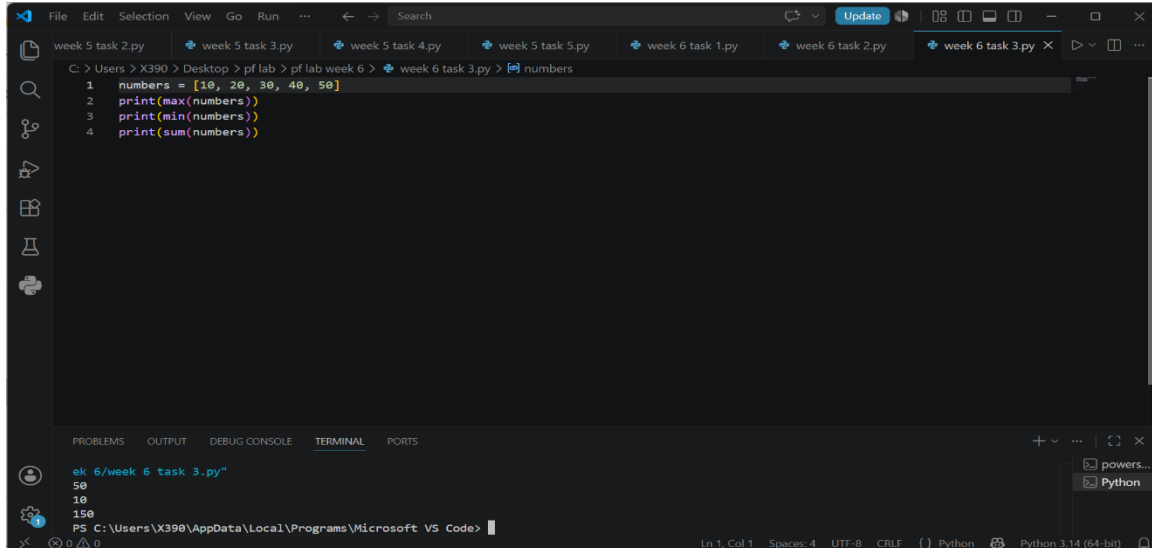
The terminal at the bottom shows the output of the script:

```
[1, 'ali', 78, 5]
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf_lab/pf_lab we
ek 6/task 2.py"
['apple', 'banana', 'mango']
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 3:

Write a program that utilizes Python's built-in mathematical functions to perform basic statistical operations on a collection of numeric data.

RESULT:



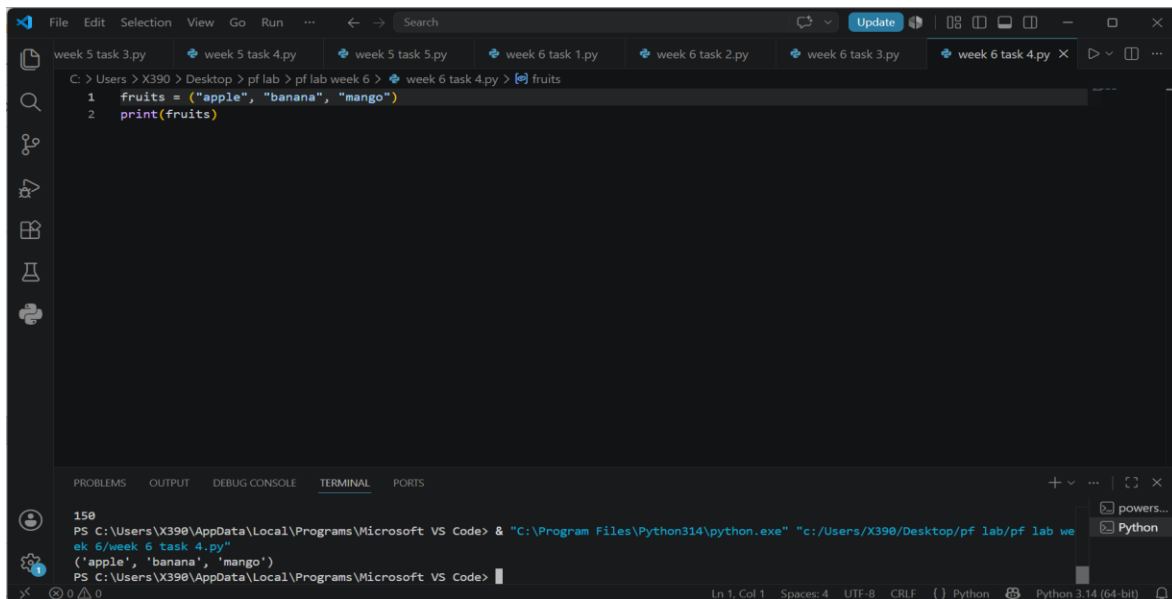
```
File Edit Selection View Go Run ... Search Update
week 5 task 2.py week 5 task 3.py week 5 task 4.py week 5 task 5.py week 6 task 1.py week 6 task 2.py week 6 task 3.py
C: > Users > X390 > Desktop > pf lab > pf lab week 6 > week 6 task 3.py > numbers
1 numbers = [10, 20, 30, 40, 50]
2 print(max(numbers))
3 print(min(numbers))
4 print(sum(numbers))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ek 6/week 6 task 3.py
50
10
150
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 4:

Write a program that demonstrates how to declare and display an immutable tuple collection in Python.

RESULT:



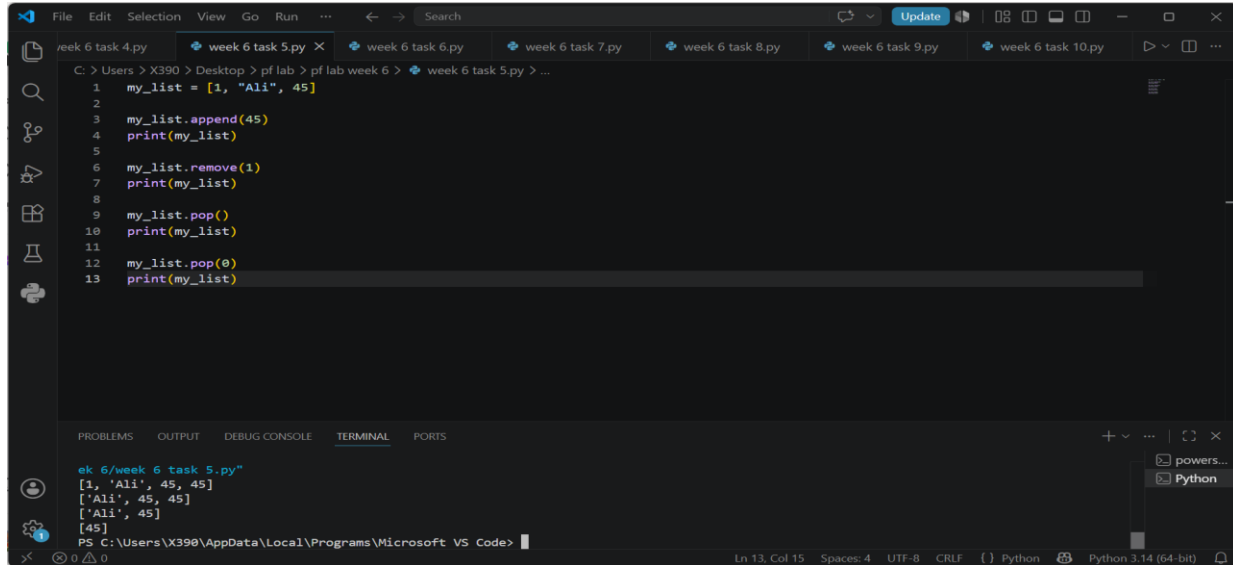
```
File Edit Selection View Go Run ... Search Update
week 5 task 3.py week 5 task 4.py week 5 task 5.py week 6 task 1.py week 6 task 2.py week 6 task 3.py week 6 task 4.py
C: > Users > X390 > Desktop > pf lab > pf lab week 6 > week 6 task 4.py > fruits
1 fruits = ("apple", "banana", "mango")
2 print(fruits)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
150
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab we
ek 6/week 6 task 4.py"
('apple', 'banana', 'mango')
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 5:

Write a program that comprehensively demonstrates structural list modification using standard built-in Python list methods.

RESULT:



The screenshot shows a VS Code editor with a Python file named 'week 6 task 5.py'. The code in the file is as follows:

```
1 my_list = [1, "Ali", 45]
2
3 my_list.append(45)
4 print(my_list)
5
6 my_list.remove(1)
7 print(my_list)
8
9 my_list.pop()
10 print(my_list)
11
12 my_list.pop(0)
13 print(my_list)
```

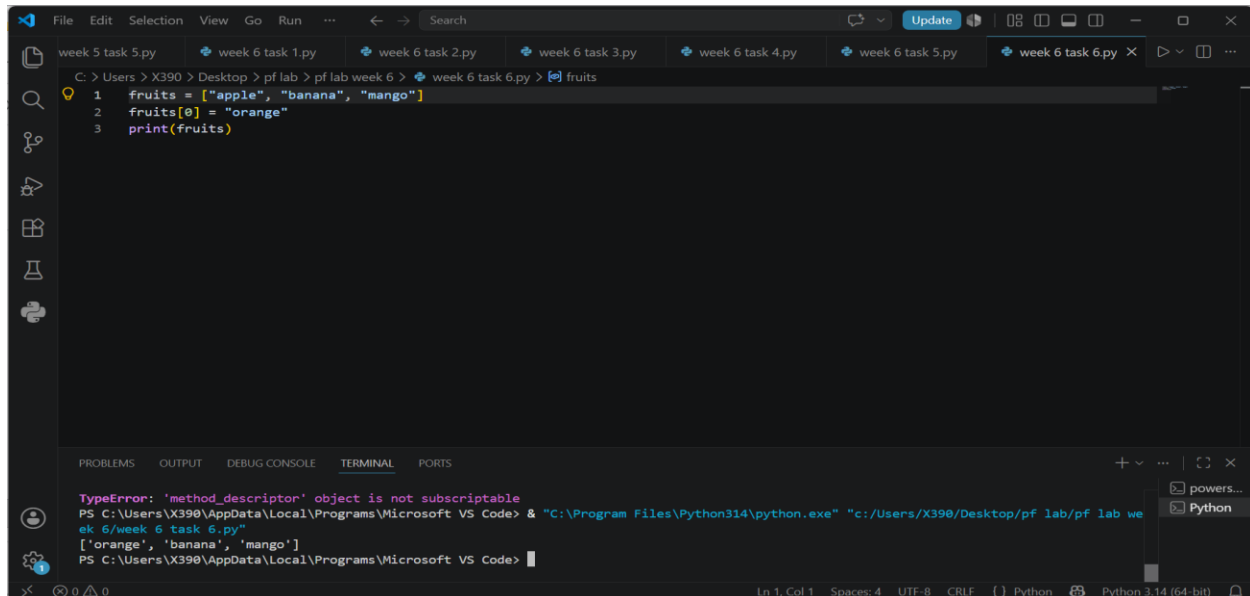
The terminal output at the bottom shows the execution results:

```
ek 6/week 6 task 5.py"
[1, 'Ali', 45, 45]
['Ali', 45, 45]
['Ali', 45]
[45]
```

TASK NO 6:

Write a program that demonstrates the concept of mutability in Python lists by modifying an item at a specific index.

RESULT:



The screenshot shows a VS Code editor with a Python file named 'week 6 task 6.py'. The code in the file is as follows:

```
1 fruits = ["apple", "banana", "mango"]
2 fruits[0] = "orange"
3 print(fruits)
```

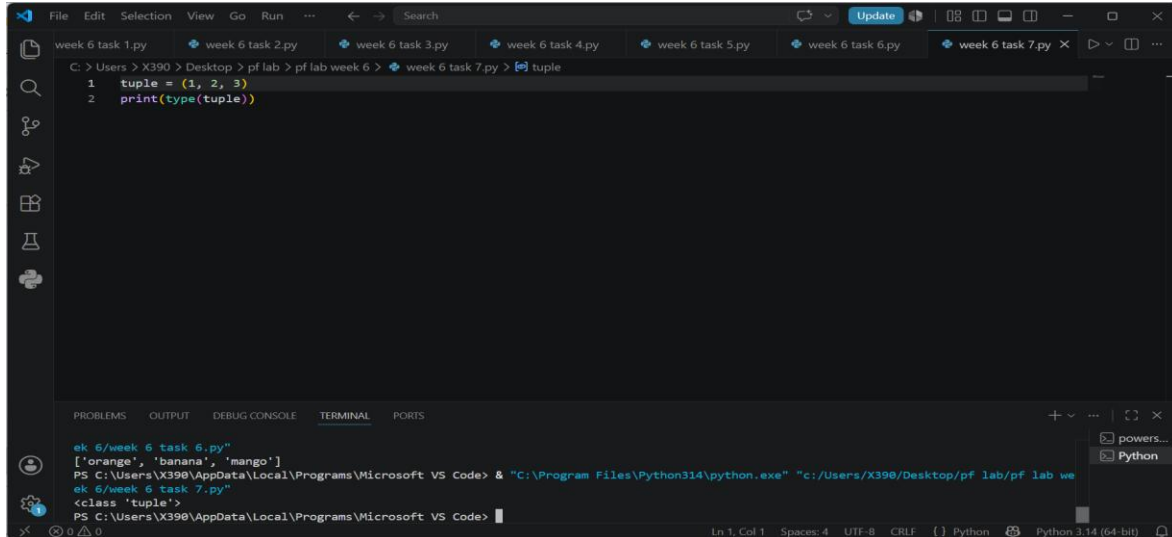
The terminal output shows a runtime error:

```
TypeError: 'method_descriptor' object is not subscriptable
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab we
ek 6/week 6 task 6.py"
['orange', 'banana', 'mango']
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code>
```

TASK NO 7:

Write a program that demonstrates how to check the class type of a data structure dynamically in Python.

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 6 task 7.py' open. The code in the editor is:

```
1 tuple = (1, 2, 3)
2 print(type(tuple))
```

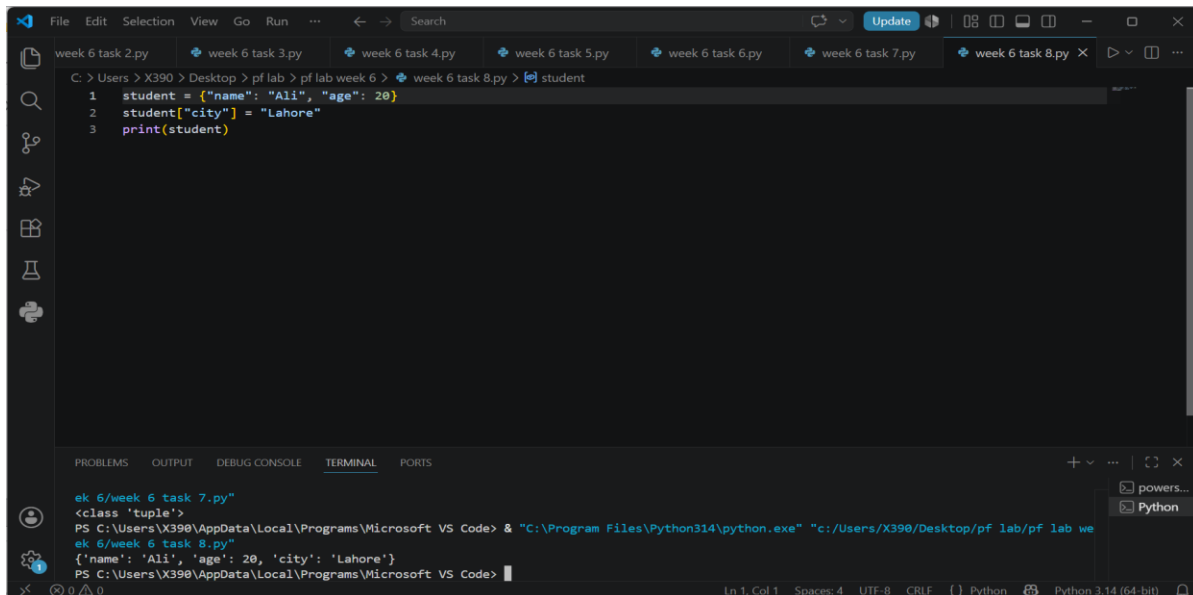
The terminal at the bottom shows the command prompt output:

```
PS C:\Users\X390\Desktop\pf lab > python week 6 task 7.py
<class 'tuple'>
```

TASK NO 8:

Write a program that demonstrates how to initialize a Python dictionary and dynamically insert a new key-value pair into the existing structure

RESULT:



The screenshot shows the Visual Studio Code editor with a file named 'week 6 task 8.py' open. The code in the editor is:

```
1 student = {"name": "Ali", "age": 20}
2 student["city"] = "Lahore"
3 print(student)
```

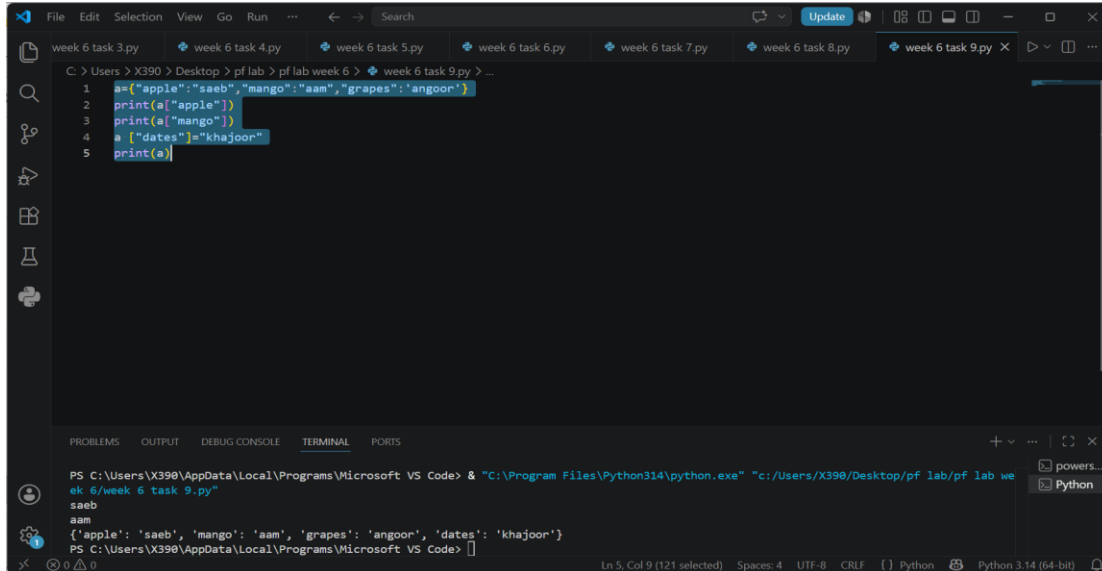
The terminal at the bottom shows the command prompt output:

```
PS C:\Users\X390\Desktop\pf lab > python week 6 task 8.py
{'name': 'Ali', 'age': 20, 'city': 'Lahore'}
```

TASK NO 9:

Write a program that demonstrates how to access mapped values via their keys and dynamically insert new key-value pairs into a Python dictionary.

RESULT:



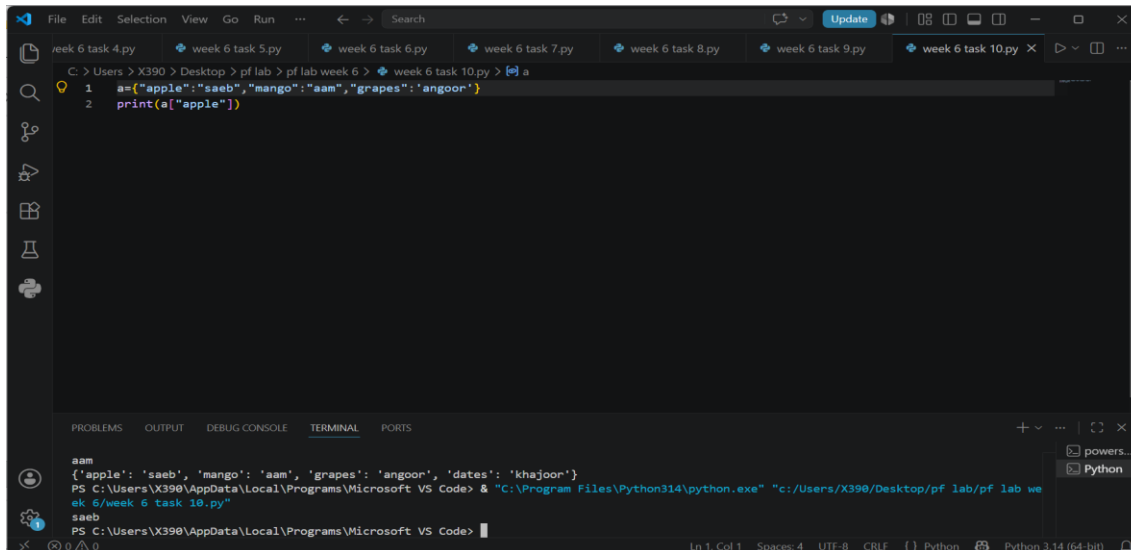
```
File Edit Selection View Go Run ... Search
week 6 task 3.py week 6 task 4.py week 6 task 5.py week 6 task 6.py week 6 task 7.py week 6 task 8.py week 6 task 9.py
C:\Users\X390\Desktop> pf lab > pf lab week 6 > week 6 task 9.py > ...
1 a={"apple":"saeb","mango":"aam","grapes":"angoor"}
2 print(a["apple"])
3 print(a["mango"])
4 a["dates"]="khajoor"
5 print(a)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\X390\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/X390/Desktop/pf lab/pf lab we
ek 6/week 6 task 9.py"
saeb
aam
{'apple': 'saeb', 'mango': 'aam', 'grapes': 'angoor', 'dates': 'khajoor'}
```

TASK NO 10:

Write a program that demonstrates how to isolate and retrieve a specific value from an associative dictionary collection using its corresponding key.

RESULT:



```
File Edit Selection View Go Run ... Search
week 6 task 4.py week 6 task 5.py week 6 task 6.py week 6 task 7.py week 6 task 8.py week 6 task 9.py week 6 task 10.py
C:\Users\X390\Desktop> pf lab > pf lab week 6 > week 6 task 10.py > a
1 a={"apple":"saeb","mango":"aam","grapes":"angoor"}
2 print(a["apple"])

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
aam
{'apple': 'saeb', 'mango': 'aam', 'grapes': 'angoor', 'dates': 'khajoor'}
```

CONCLUSION:

- Managing local and global variables effectively is crucial for controlling data access and structural integrity across a program. Local variables keep the main program space clean by securely restricting temporary data inside individual functions. Their short lifespan ensures that system memory is immediately reclaimed as soon as a function finishes executing. Global variables provide a persistent state throughout the script's entire lifecycle, allowing distinct modules to share configuration data. However, overusing global scopes can introduce silent bugs and hidden dependencies that make codebases incredibly difficult to debug. Ultimately, mastering the balance between these two scopes allows developers to build robust, secure, and highly maintainable software systems.